PROYECTO 1

TacticalGrid

Juego táctico 2D con búsqueda y toma de decisiones

Sistemas Inteligentes I

Primer corte · Proyecto

Ingeniería de Sistemas y Computación
UCALDAS

El juego es el entorno de experimentación.
El proyecto es el sistema inteligente que decide dentro de él.

Propósito y escenario

En este proyecto construirán un juego táctico bidimensional por turnos en el que dos bandos compiten sobre un mapa visto desde arriba. En una zona neutral se encuentra un recurso estratégico. Cada equipo intentará llegar hasta él, tomarlo y transportarlo hasta su propia base antes que el adversario.

El mapa tendrá obstáculos, diferentes tipos de terreno y varias unidades por bando. Algunos terrenos serán más costosos de atravesar que otros y, a medida que avance la partida, las decisiones del adversario podrán modificar cuál es la alternativa más conveniente.

La intención no es desarrollar un videojuego comercial. La interfaz debe permitir observar y experimentar con la inteligencia del sistema; los gráficos, animaciones y efectos visuales no son el centro de la evaluación.

A lo largo del proyecto trabajarán sobre un mismo problema, pero la pregunta irá cambiando: primero buscarán una ruta, después una ruta de menor costo, luego intentarán aprovechar conocimiento del entorno para orientar la búsqueda y, finalmente, deberán decidir teniendo en cuenta que existe otro jugador que también razona y actúa.

Alcance y decisiones de diseño

Hay reglas que serán comunes para todos los equipos y otras que quedarán abiertas para que cada grupo tome decisiones propias. Las decisiones abiertas deberán quedar implementadas, documentadas y sustentadas.

- Hay libertad para que decidan cómo estructurar internamente el software.
- Debe existir una separación clara entre la lógica del juego, la representación del escenario, los algoritmos de búsqueda/decisión y la interfaz gráfica.
- La representación interna del estado, las estructuras auxiliares, la segunda heurística, la función de utilidad y ciertos detalles de las unidades son decisiones del equipo.
- Los algoritmos no pueden depender de un mapa particular ni de posiciones escritas directamente en el código.
- La solución deberá aceptar escenarios externos suministrados por el profesor mediante el formato JSON común definido en este documento.

Escenario de juego

El escenario será una cuadrícula bidimensional. Los mapas utilizados durante el desarrollo deberán tener como mínimo 20 × 20 posiciones, aunque la aplicación deberá admitir otras dimensiones válidas suministradas mediante el archivo de escenario.

Elemento	Regla mínima
Camino	Terreno transitable de costo bajo.
Terreno regular	Terreno transitable de costo intermedio.
Terreno difícil	Terreno transitable de costo mayor.
Obstáculo	Celda no transitable.
Base A / Base B	Zona de llegada de cada bando.
Recurso	Objetivo estratégico de la partida.

Los costos no estarán codificados de manera fija dentro de los algoritmos: deberán obtenerse del escenario cargado. Deben existir mapas en los que una ruta con menos movimientos resulte más costosa que una ruta más larga.

Bandos, unidades y turnos

Cada bando controlará al menos tres unidades. El equipo podrá trabajar con unidades idénticas o proponer tipos diferentes, siempre que esas diferencias tengan una función clara dentro del problema y no conviertan el proyecto en un sistema de combate excesivamente complejo.

El juego será por turnos. En cada turno un bando seleccionará una unidad y realizará una acción válida. No se requiere concurrencia, movimiento simultáneo ni lógica de tiempo real.

Ambos bandos deben operar bajo las mismas reglas. MAX y MIN no representan inteligencias distintas ni niveles diferentes de capacidad; son roles dentro del árbol Minimax.

Objetivo de la partida

El recurso estratégico estará inicialmente en una zona neutral. Una unidad podrá recogerlo al alcanzar su posición y, mientras lo transporte, deberá intentar llegar a la base de su equipo. Un bando gana cuando consigue llevar el recurso hasta su propia base.

El equipo deberá definir reglas deterministas para situaciones como bloqueos, imposibilidad de continuar, ausencia de una ruta válida o pérdida del portador del recurso. Estas reglas deberán ser coherentes con el modelo de estados y podrán ser verificadas durante la sustentación.

Modelado del problema y del estado

El modelado es parte central del proyecto. Antes de pensar en los algoritmos, el equipo deberá establecer qué información define completamente una situación del juego y qué acciones permiten pasar de un estado a otro.

En un ejercicio básico de navegación podría bastar con representar un estado mediante la posición (fila, columna). Sin embargo, cuando aparecen varias unidades, turnos, posesión del recurso y acciones adversariales, esa representación deja de ser suficiente.

Dos estados pueden contener una unidad en la misma casilla y, aun así, ser estados diferentes: por ejemplo, si en uno la unidad transporta el recurso y en el otro no.

La solución deberá permitir identificar de manera inequívoca el estado inicial, las acciones válidas, los estados sucesores, la condición objetivo y el costo de las acciones. El equipo deberá explicar por qué su representación contiene la información suficiente para tomar decisiones correctas.

Búsqueda no informada

En una primera etapa se ignorará temporalmente el comportamiento del adversario. Una unidad deberá desplazarse desde una posición inicial hasta un objetivo. Sobre el mismo modelo de escenario deberán implementar BFS y DFS.

Métrica	Qué debe registrar
Solución encontrada	Sí / No.
Longitud	Número de movimientos del camino.
Estados generados	Estados incorporados a la búsqueda.
Estados expandidos	Estados cuyos sucesores fueron analizados.
Máximo de frontera	Mayor número de estados almacenados simultáneamente.
Tiempo	Tiempo de ejecución medido por la aplicación.

Búsqueda basada en costos

Después se incorporarán los costos del terreno y deberán implementar Búsqueda de Costo Uniforme (UCS). El objetivo será minimizar el costo acumulado y no necesariamente la cantidad de movimientos.

El equipo deberá preparar al menos un escenario donde BFS y UCS produzcan caminos diferentes y deberá explicar el motivo de esa diferencia a partir de los costos del mapa.

Búsqueda informada

Deberán implementar A* utilizando f(n) = g(n) + h(n). El equipo trabajará con al menos dos heurísticas. Una podrá ser distancia Manhattan; la segunda deberá ser propuesta y justificada por el equipo.

- Qué intenta estimar la heurística.
- Qué información del estado utiliza.
- Cuál es su costo computacional.
- Si se considera admisible y por qué.
- En qué escenarios puede orientar bien la búsqueda y en cuáles puede perder capacidad de discriminación.

Búsqueda con memoria limitada

También deberán implementar Beam Search y experimentar, como mínimo, con k = 1, 2, 4 y 8. El análisis deberá discutir el efecto de restringir la cantidad de alternativas conservadas y mostrar si esa restricción puede descartar un camino que posteriormente habría resultado conveniente.

Decisión adversarial

Cuando ambos bandos comienzan a actuar, el problema deja de ser únicamente encontrar una ruta. En determinados estados la unidad podrá avanzar, proteger una zona, bloquear una posición, aproximarse al recurso, interceptar al portador o ejecutar otras acciones definidas por el equipo.

La pregunta cambia de "¿cuál es mi mejor camino?" a "¿cuál es mi mejor acción teniendo en cuenta las respuestas posibles del adversario?".

Para resolver este componente deberán implementar Minimax y Minimax con poda alfa-beta. El árbol se explorará con profundidad limitada y la profundidad máxima deberá poder modificarse sin alterar el código.

MAX, MIN y función de utilidad

La función de utilidad se definirá desde la perspectiva de uno de los bandos. MAX intentará maximizar ese valor y MIN intentará minimizarlo. Esto no significa que MIN tenga una desventaja: disminuir la utilidad de MAX representa favorecer sus propios intereses cuando el problema se modela de manera adversarial.

Cada equipo deberá construir su propia función de evaluación. No se entregará una fórmula completa. Podrán considerar, entre otros factores, la posesión del recurso, las distancias relevantes, la proximidad del adversario, las unidades disponibles, el costo acumulado y el control de posiciones estratégicas.

Durante la sustentación deberán poder explicar por qué una acción recibe una valoración mejor que otra y cómo cambiaría el comportamiento si se modifican los pesos o criterios de la función.

Profundidad y poda alfa-beta

Minimax deberá trabajar con profundidad limitada. El equipo realizará experimentos con diferentes profundidades y analizará cómo cambian la cantidad de estados evaluados, el tiempo requerido y, cuando ocurra, la acción seleccionada.

Métrica	Minimax / Alfa-beta
Nodos generados	Cantidad total generada durante la exploración.
Nodos evaluados	Estados en los que se calculó utilidad.
Nodos podados	Ramas evitadas por alfa-beta.
Profundidad alcanzada	Profundidad efectiva de la búsqueda.
Tiempo	Tiempo de ejecución.
Acción seleccionada	Movimiento recomendado.
Valor obtenido	Valor Minimax del estado analizado.

Con la misma profundidad, función de utilidad y orden de acciones, Minimax y alfa-beta deberán producir la misma decisión. Además, el equipo deberá cambiar el orden de generación de acciones y analizar cómo afecta la cantidad de poda.

Formato común de escenarios JSON

Todos los equipos deberán implementar la carga de escenarios mediante un archivo JSON común. Esta estructura constituye un contrato de interoperabilidad: la representación interna del programa puede ser diferente, pero cualquier archivo válido que respete este formato deberá poder cargarse sin modificar el código fuente.

Durante la sustentación el profesor suministrará al menos un escenario JSON que el equipo no habrá visto previamente. La aplicación deberá cargarlo y utilizarlo directamente.

Campos obligatorios

Campo	Descripción
version	Versión del formato de escenario.
mapa.filas / mapa.columnas	Dimensiones de la cuadrícula.
tipos_terreno	Catálogo de terrenos, costos y transitabilidad.
terreno	Matriz que describe cada celda del mapa.
bases.A / bases.B	Posiciones de las bases.
recurso	Posición del recurso cuando no está siendo transportado.
unidades	Listado de unidades, bando, tipo y posición.
turno	Bando que debe actuar.
juego.portador_recurso	Identificador de la unidad que actualmente transporta el recurso. Su valor será null cuando el recurso no esté en posesión de ninguna unidad.
prueba	Configuración opcional para ejecutar una prueba concreta.

Ejemplo de escenario

```
{
 "version": "1.0",
 "mapa": {
  "filas": 4,
  "columnas": 4
 },
 "tipos_terreno": {
  "camino": {"costo": 1, "transitable": true},
  "pasto": {"costo": 2, "transitable": true},
  "bosque": {"costo": 4, "transitable": true},
  "pantano": {"costo": 7, "transitable": true},
  "muro": {"transitable": false}
 },
 "terreno": [
  ["camino", "camino", "pasto", "pasto"],
  ["camino", "muro", "bosque", "pasto"],
  ["camino", "camino", "pantano", "pasto"],
  ["pasto", "pasto", "camino", "camino"]
 ],
 "bases": {
  "A": {"fila": 0, "columna": 0},
  "B": {"fila": 3, "columna": 3}
 },
 "recurso": {"fila": 2, "columna": 2},
 "unidades": [
  {"id": "A1", "bando": "A", "tipo": "estandar", "fila": 0, "columna": 1},
  {"id": "A2", "bando": "A", "tipo": "estandar", "fila": 1, "columna": 0},
  {"id": "B1", "bando": "B", "tipo": "estandar", "fila": 3, "columna": 2},
  {"id": "B2", "bando": "B", "tipo": "estandar", "fila": 2, "columna": 3}
 ],
 "turno": "A",
 "juego": {
  "portador_recurso": null,
  "profundidad_maxima_minimax": 4
 },
 "prueba": {
  "modo": "busqueda",
  "unidad_inicio": "A1",
  "objetivo": {"fila": 3, "columna": 2}
 }
}
```

Los nombres de campos y valores definidos por esta especificación deberán respetarse. Los equipos podrán incluir campos adicionales propios siempre que no alteren ni cambien el significado de los campos obligatorios.

Modos de prueba

El campo prueba permitirá que el profesor cargue un estado y solicite una operación concreta. Como mínimo deberán contemplarse los siguientes modos:

Modo	Propósito
busqueda	Resolver una navegación concreta con BFS, DFS, UCS, A* o Beam Search.
partida	Cargar el escenario completo para continuar el juego por turnos.
adversarial	Evaluar un estado mediante Minimax o alfa-beta y seleccionar una acción.

Validación del escenario

La carga deberá ser completa o no aplicarse. Antes de reemplazar el escenario actual, el sistema deberá validar el archivo y explicar con claridad cualquier error encontrado.

- Filas y columnas positivas y consistentes con la matriz;
- Todos los terrenos utilizados existen en tipos_terreno;
- Los terrenos transitables tienen un costo positivo;
- Bases, recurso y unidades se encuentran dentro del mapa;
- Ninguna unidad se encuentra sobre una celda no transitable;
- Los identificadores de unidad son únicos;
- Cada unidad pertenece a a o b;
- El turno corresponde a a o b;
- Si portador_recurso no es null, identifica una unidad existente.

Un escenario inválido deberá rechazarse sin dejar el sistema en un estado parcialmente modificado.

Compatibilidad con escenarios externos

La aplicación deberá comportarse correctamente con escenarios válidos que cambien dimensiones, distribución de terrenos, costos, obstáculos, posiciones de las bases, unidades y recurso.

Los costos deberán tomarse del JSON y no de constantes dispersas dentro de los algoritmos. Por ejemplo, el profesor podrá cambiar el costo del pantano de 7 a 15 y solicitar que el equipo anticipe y luego observe el efecto sobre UCS y A*.

No se realizarán modificaciones de código para adaptar la aplicación al mapa suministrado durante la sustentación.

Resultados y métricas comparables

La interfaz deberá mostrar de forma clara las métricas utilizadas para comparar algoritmos. Para una prueba de búsqueda, la información mínima será equivalente a la siguiente estructura:

```
{
 "algoritmo": "A_ESTRELLA",
 "exito": true,
 "camino": [[3,2], [3,3], [4,3], [5,3]],
 "costo": 9,
 "estados_generados": 42,
 "estados_expandidos": 21,
 "maximo_frontera": 13
}
```

No es obligatorio exportar cada resultado en JSON, pero la aplicación deberá hacer accesible información equivalente para que los experimentos puedan reproducirse y compararse.

Visualización

La interfaz gráfica deberá permitir entender qué está ocurriendo. Como mínimo mostrará la cuadrícula, tipos de terreno, obstáculos, bases, recurso, unidades, bando de cada unidad, portador del recurso y turno actual. Cuando se ejecute una búsqueda deberá poder verse el camino encontrado.

Se valorará la posibilidad de observar estados explorados, fronteras o decisiones consideradas, pero la calidad artística no será un criterio importante de evaluación.

Verificación de soluciones

La aplicación deberá incorporar una opción de verificación. El objetivo es que el sistema no solo produzca una respuesta, sino que pueda comprobar propiedades básicas de esa respuesta.

- Cada movimiento del camino es válido;
- Ninguna unidad atraviesa obstáculos;
- El costo reportado coincide con las transiciones realizadas;
- El estado final satisface la condición objetivo;
- No se generan estados inválidos;
- Las acciones consideradas por minimax son legales dentro de las reglas del juego.

Documentación técnica de métodos

Los métodos que implementen lógica relevante deberán contar con documentación con enfoque de ingeniería. No se considera suficiente un comentario que únicamente diga qué hace el método.

Elemento	Contenido esperado
Purpose	Responsabilidad concreta del método dentro de la solución.
Preconditions	Condiciones que deben cumplirse antes de ejecutarlo.
Postconditions	Condiciones que deben cumplirse después de una ejecución correcta.
Complexity	Costo temporal y, cuando corresponda, espacial.
AI usage	Indicar explícitamente Yes o No.
AI intervention	Si se utilizó IA, describir con precisión en qué parte intervino.
Student validation	Explicar cómo se revisó, probó, modificó o validó el aporte generado con IA.

Esta documentación será obligatoria, especialmente, para algoritmos de búsqueda, heurísticas, generación de sucesores, cálculo de costos, funciones de utilidad, Minimax, alfa-beta y validadores. No se exige este nivel de documentación para métodos triviales.

La documentación deberá corresponder a la versión real del método. Si describe una solución distinta de la implementada, se considerará documentación inconsistente.

Uso de inteligencia artificial generativa

La IA generativa podrá utilizarse como herramienta de apoyo. Su uso no disminuye por sí mismo la calificación, pero deberá declararse. Cada estudiante será responsable de todo el código que aparezca en la entrega y deberá estar en capacidad de explicarlo, modificarlo y defenderlo.

Una funcionalidad que opere correctamente, pero que el estudiante no pueda explicar, justificar o adaptar durante la sustentación, no demostrará dominio suficiente del resultado entregado.

La trazabilidad del uso de IA se verificará tanto en la documentación de métodos como en el informe técnico. La omisión deliberada del uso realizado se considerará una inconsistencia en la documentación del desarrollo.

Casos mínimos para demostrar la solución

Caso	Qué debe demostrarse
1. BFS frente a DFS	Ambos encuentran solución, pero recorren el espacio de búsqueda de manera diferente.
2. Ruta corta vs. ruta económica	BFS y UCS producen soluciones distintas debido al costo del terreno.
3. Obstáculo y heurística	El objetivo parece cercano, pero una barrera obliga a rodearlo.
4. Comparación de heurísticas	A* se ejecuta con las dos heurísticas sobre el mismo escenario.
5. Beam Search	Cambiar k modifica la exploración y, cuando sea posible, la solución.

6. Estado compuesto	La misma posición física representa estados diferentes por información adicional del juego.
7. Decisión adversarial	La mejor decisión según Minimax no coincide necesariamente con avanzar directamente al recurso.
8. Minimax vs. alfa-beta	Seleccionan la misma acción y alfa-beta evita evaluar parte del árbol.
9. Orden de acciones	Cambiar el orden de sucesores modifica la cantidad de poda.
10. Profundidad	El mismo estado se evalúa con distintas profundidades y se analizan los efectos.

Cada equipo deberá preparar datos reproducibles, declarar el estado inicial y comparar los resultados esperados con los obtenidos. Estos casos no reemplazan las pruebas externas realizadas por el profesor.

Pruebas de comprensión y adaptación

Durante la sustentación el profesor podrá introducir modificaciones no conocidas previamente por el equipo. El propósito no es agregar funcionalidades nuevas, sino comprobar que los estudiantes comprenden la solución que entregaron y pueden razonar sobre su comportamiento.

- Cambiar costos de terrenos en el JSON;
- Mover obstáculos, bases, unidades o recurso;
- Cambiar el objetivo de una prueba de navegación;
- Modificar la profundidad de Minimax;
- Cambiar el orden de generación de acciones;
- Solicitar la ejecución de otro algoritmo sobre el mismo escenario;
- Pedir una predicción razonada del comportamiento antes de ejecutar;
- Seleccionar un método al azar y solicitar una modificación pequeña en el código.

Entregables

- Repositorio Git que permita evidenciar los aportes de cada integrante. Debe incluir un Readme.md donde, como mínimo, estén los nombres de los integrantes del equipo
- Código fuente completo e instrucciones de instalación y ejecución.
- Escenarios JSON utilizados durante el desarrollo y casos mínimos de prueba.
- Manual de usuario.
- Documento técnico con: modelado del problema, representación de estados, reglas del juego, costos, heurísticas, función de utilidad, metodología experimental, resultados, análisis de complejidad, limitaciones y reporte del uso de IA.
- Evidencia suficiente para reproducir los experimentos reportados.

El repositorio y los documentos deberán corresponder a la misma versión presentada durante la sustentación.

Sustentación

La sustentación tendrá un componente individual. Cada integrante deberá estar en capacidad de explicar cualquier parte relevante de la solución, aunque no haya sido el autor principal de ese archivo o método.

- Explicar la representación de un estado;
- Ejecutar bfs, dfs, ucs, a* o beam search sobre un escenario externo;
- Justificar una heurística y discutir su admisibilidad;
- Explicar una decisión de minimax y la función de utilidad;
- Explicar por qué min no está en desventaja frente a max;
- Analizar el efecto de cambiar la profundidad o el orden de acciones;

- Explicar la complejidad de un método;
- Explicar el uso de ia declarado y la validación realizada;
- Realizar una modificación pequeña si se solicita.

Evaluación

La evaluación se distribuirá en 50 % para la funcionalidad y calidad de la solución desarrollada por el equipo y 50 % para la sustentación individual. La valoración individual podrá afectar la nota final del estudiante si se evidencia una diferencia importante entre el funcionamiento del producto y su dominio real de la solución.

Componente del 50 % grupal	Valoración %
Modelado del problema y representación de estados	15 %
BFS, DFS y Costo Uniforme	15 %
A*, heurísticas y Beam Search	20 %
Minimax y poda alfa-beta	20 %
Experimentación, métricas e interpretación	10 %
Interfaz y visualización	5 %
Validación de escenarios y soluciones	5 %
Calidad y documentación técnica del código	10 %
Total	100 %

Pregunta central del proyecto

¿Cómo cambia la forma de decidir de un sistema inteligente cuando pasa de encontrar una ruta, a optimizar su costo, utilizar conocimiento heurístico y finalmente actuar frente a un adversario que también toma decisiones?

El proyecto estará bien resuelto cuando el equipo no solo pueda mostrar que sus algoritmos funcionan, sino también explicar por qué se comportan como lo hacen, qué supuestos sostienen sus decisiones y cómo reaccionan frente a un escenario que no habían visto antes.